

REPORT

Title

Minimax Algorithm for Connect Four

Aim

To implement the **Minimax algorithm** for a Connect Four game in Python and enable the computer to select the best possible move.

Problem Statement

Connect Four is a two-player game played on a 6×7 board. Players take turns dropping their pieces into columns. The piece occupies the lowest available position in the selected column. A player wins by forming four consecutive pieces horizontally, vertically, or diagonally.

The objective is to implement an intelligent computer player using the Minimax algorithm that evaluates possible moves and selects the most favorable move.

Requirements

- Python 3.x
- Python IDLE / VS Code / any Python IDE
- Basic knowledge of Python
- Knowledge of game trees and the Minimax algorithm

Procedure

1. Create a 6×7 Connect Four board.
2. Represent the human player using x and the computer using o.
3. Display the board to the user.

4. Accept a valid column from the human player.
5. Place the human player's piece in the lowest available position.
6. Check whether the human player has formed four consecutive pieces.
7. Generate all possible moves for the computer.
8. Apply the Minimax algorithm to evaluate the possible moves.
9. Use the maximizing step for the computer and the minimizing step for the human.
10. Assign scores to winning, losing, and possible board positions.
11. Select the move with the highest score for the computer.
12. Repeat the process until a player wins or the board becomes full.

Program

```
ROWS = 6
COLS = 7

HUMAN = "X"
AI = "O"
EMPTY = "."

def create_board():
    return [[EMPTY for _ in range(COLS)] for _ in range(ROWS)]

def print_board(board):
    print()
    for row in board:
        print(" ".join(row))
    print("0 1 2 3 4 5 6")

def is_valid_move(board, col):
    return board[0][col] == EMPTY

def get_valid_moves(board):
    return [col for col in range(COLS) if is_valid_move(board, col)]

def get_next_row(board, col):
    for row in range(ROWS - 1, -1, -1):
        if board[row][col] == EMPTY:
            return row
    return -1

def winning_move(board, piece):
    for row in range(ROWS):
        for col in range(COLS - 3):
            if all(board[row][col+i] == piece for i in range(4)):
                return True
```

```

for row in range(ROWS - 3):
    for col in range(COLS):
        if all(board[row+i][col] == piece for i in range(4)):
            return True

for row in range(ROWS - 3):
    for col in range(COLS - 3):
        if all(board[row+i][col+i] == piece
                for i in range(4)):
            return True

for row in range(3, ROWS):
    for col in range(COLS - 3):
        if all(board[row-i][col+i] == piece
                for i in range(4)):
            return True

return False

def score_position(board):
    score = 0

    for row in board:
        score += row.count(AI)

    return score

def minimax(board, depth, maximizing_player):

    valid_moves = get_valid_moves(board)

    if winning_move(board, AI):
        return None, 1000000

    if winning_move(board, HUMAN):
        return None, -1000000

    if not valid_moves or depth == 0:
        return None, score_position(board)

    if maximizing_player:
        best_score = -float("inf")
        best_col = valid_moves[0]

        for col in valid_moves:
            row = get_next_row(board, col)
            board[row][col] = AI

            score = minimax(board, depth - 1, False)[1]

            board[row][col] = EMPTY

            if score > best_score:
                best_score = score
                best_col = col

    return best_col, best_score

```

```

else:
    best_score = float("inf")
    best_col = valid_moves[0]

    for col in valid_moves:
        row = get_next_row(board, col)
        board[row][col] = HUMAN

        score = minimax(board, depth - 1, True)[1]

        board[row][col] = EMPTY

        if score < best_score:
            best_score = score
            best_col = col

    return best_col, best_score

```

Sample Output

```

CONNECT FOUR
You = X
Computer = O

```

```

. . . . .
. . . . .
. . . . .
. . . . .
. . . . .
. . O X . .
0 1 2 3 4 5 6

```

```

Enter column (0-6): 3
Computer chooses column: 2

```

```

...

```

```

Computer wins!

```

Conclusion

The Connect Four game was successfully implemented using the **Minimax algorithm**. The algorithm evaluates possible future moves and allows the computer to choose a strategically favorable move. Thus, Minimax provides an effective approach for implementing an intelligent game-playing agent.